

КАФЕДРА ВЫЧИСЛИТЕЛЬНЫХ СИСТЕМ И СЕТЕЙ

КУРСОВАЯ РАБОТА
ЗАЩИЩЕНА С ОЦЕНКОЙ
РУКОВОДИТЕЛЬ

преподаватель
должность, уч. степень, звание

подпись, дата

Е. Е. Майн
инициалы, фамилия

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К КУРСОВОЙ РАБОТЕ

Практический курс по химии в
виртуальной реальности (VR)

по дисциплине: Компьютерная графика

РАБОТУ ВЫПОЛНИЛИ

СТУДЕНТЫ ГР. 4143

подпись, дата

В. Д. Алексеева
А. М. Гридин
П. В. Суслов
С. А. Шалев
инициалы, фамилия

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1. Постановка задачи	4
2. Спецификация	9
3. Текст программы	11
3.1. Текст программы	14
3.2. Рисунки моделей.....	25
4. Описание программы	34
4.1. Общие сведения	37
4.2. Функциональное назначение.....	37
4.3. Описание логической структуры	37
4.4. Используемые технические средства	38
4.5. Вызов и загрузка	38
5. Описание применения	39
5.1. Назначение программы	42
5.2. Условие применения	42
5.3. Описание задачи	42
5.4. Характеристики занимаемой памяти.....	43
ЗАКЛЮЧЕНИЕ	44
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	45

ВВЕДЕНИЕ

Данная работа предназначена для закрепления учебного материала, изученного по курсу «Компьютерная графика».

Программа представляет собой реализованное VR-приложение «Практический курс по химии». Управление игрой простое, интерфейс интуитивно понятен. Программа прекрасно подходит для начинающих водителей, которые боятся сломать настоящую машину.

Цель курсовой работы – выработать навыки использования алгоритмов для решения конкретных задач, закрепления знаний по программированию на языке *C#* и по моделированию в приложении Blender.

Кроме этого, курсовая работа предназначена для приобретения навыков оформления сопроводительной документации к разработанному программному средству.

1. ПОСТАНОВКА ЗАДАЧИ

НАЗНАЧЕНИЕ РАЗРАБОТКИ

Курсовая работа VR-приложение «Практический курс по химии»:

- является комплексным проектом, охватывающим различные аспекты разработки ПО;
- содержит все основные аспекты компьютерной игры – симулятора;
- является продуктом сферы компьютерных развлечений.

ТРЕБОВАНИЯ К ПРОГРАММЕ

ТРЕБОВАНИЯ К ФУНКЦИОНАЛЬНЫМ ХАРАКТЕРИСТИКАМ

В процессе работы приложения пользователь является непосредственным участником игрового процесса и оказывает непосредственное влияние на него.

Программа должна обладать следующим функционалом:

- а) внутриигровой функционал:
 - 1) система выбора опыта;
 - 2) система взятия предметов;
 - 3) система взаимодействия игровых объектов;
- б) интерфейс пользователя:
 - 1) главное меню;
 - 2) графический интерфейс пользователя.

ТРЕБОВАНИЯ К ВХОДНЫМ И ВЫХОДНЫМ ДАННЫМ

Входными данными в компьютерной игре являются игровые настройки пользователя, а также непосредственное управление во время игрового процесса с помощью VR шлема и контроллеров. Программа относится к играм в реальном времени, действия игрока незамедлительно оказывают влияние на игровой процесс.

Выходными данными являются графическая и текстовая интерпретация игрового процесса в шлеме игрока. Действия игрока влияют на игровой процесс и текущее состояние игровой сцены и интерфейса.

ТРЕБОВАНИЯ К НАДЕЖНОСТИ

В программе должна присутствовать проверка входной информации на соответствие типов, принадлежность диапазону допустимых значений и соответствие структурной корректности.

ТРЕБОВАНИЯ К СОСТАВУ И ПАРАМЕТРАМ ТЕХНИЧЕСКИХ СРЕДСТВ

Минимальные системные требования:

- ОС (операционная система): Windows 10;
- Процессор: Линейка Intel Core i3, i5, i7, AMD FX 8000;
- Оперативная память: 16 Гб;
- Жесткий диск: 5 Гб свободно;
- Видео карта: Видеокарта уровня GT1070 и выше;
- DirectX 10;
- VR набор Oculus.

Рекомендуемые системные требования:

- ОС (операционная система): Windows 11;
- Процессор: Процессор уровня Intel Core i7-10700F
- Оперативная память: 16 Гб;
- Жесткий диск: 128 Гб свободно;
- Видео карта: Видеокарта уровня RTX 3060 и выше;
- DirectX 11;
- VR комплект Oculus.

ТРЕБОВАНИЯ К ИНФОРМАЦИОННОЙ И ПРОГРАММНОЙ СОВМЕСТИМОСТИ

Программа должна функционировать под управлением ОС семейства Windows следующих версий: Windows 10, 11. В приложении используются библиотеки платформы .NET Framework. Также требуется установленный DirectX 10 или более поздней версии.

ТРЕБОВАНИЯ К МЕХАНИКЕ ИГРОВОГО ПРОЦЕССА

Игровой процесс разбит на три этапа:

- подготовительный;
- основной;
- заключительный

ПОДГОТОВИТЕЛЬНЫЙ ЭТАП

Подготовительный этап заключается в том, что при входе в приложение на экране всплывает окно “Меню”, в котором можно выбрать необходимый к выполнению опыт, выйти из приложения или настроить некоторые параметры.

ОСНОВНОЙ ЭТАП

Основной этап содержит в себе основной игровой процесс. После выбора опыта перед пользователем появится стол, со всем необходимым оборудованием. Также откроется окно с текстом инструкций к проведению эксперимента.

В качестве работ должны быть представлены следующие элементы:

- Опыт на эффект хемилюминесценции;
- Реакция раствора сульфата меди и раствора аммиака с выпадением осадка;
- Опыт “дым без огня”;
- Окрашивание гвоздя в растворе медного купороса;
- Эксперимент с сухим льдом;
- Опыт “Вулкан”;
- Реакция цезия и воды;
- Эксперимент с бертолетовой солью и красным фосфором;

Управление происходит посредством использования VR шлема и контроллеров.

ЗАКЛЮЧИТЕЛЬНЫЙ ЭТАП

Заключительный этап наступает в случае неправильного выполнения эксперимента. Тогда пользователь возвращается к экрану меню для повторного выбора опыта.

ТРЕБОВАНИЯ К ДИЗАЙНУ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА

Пользовательский интерфейс должен быть интуитивно понятным для среднестатистического пользователя компьютера (или иной ЭВМ). Текст должен быть хорошо читаем, информация должна быть краткой и соответствующей сущности элементов интерфейса. Интерфейс должен быть

представлен в пастельных оттенках, комфортных для длительного восприятия человеческим зрением. Световые и прочие эффекты не должны содержать резкий световых вспышек и резкой смены яркости. Интерфейс должен быть выполнен в преимущественно светлых оттенках и придерживаться единой стилистики.

ИСПОЛЬЗУЕМОЕ ПРОГРАМНОЕ ОБЕСПЕЧЕНИЕ

- Программа реализована на игровом движке Unity 3D в рамках программного пакета Unity Hub;
- Язык программирования – C#;
- Среда 3D-моделирования – Blender;

44.4143.22-01

9

2.1 Спецификация

Обозначение	Наименование	Примечание
44.4143.22-12	Документация Программа Практический курс по химии в виртуальной реальности (VR) Текст программы	
44.4143.22-13	Документация Программа Практический курс по химии в виртуальной реальности (VR) Описание программы	
44.4143.22-31	Документация Программа Практический курс по химии в виртуальной реальности (VR) Описание применения	

АННОТАЦИЯ

В документе приводится текст программы «Практический курс по химии», написанной на языке C#, а также результат отладки и тестирования.

СОДЕРЖАНИЕ

3.1. Текст программы

3.2. Рисунки моделей

3.1. ТЕКСТ ПРОГРАММЫ

Файл «board.cs»

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;

/// Include the name space for TextMesh Pro
using TMPro;

public class board : MonoBehaviour
{
    public GameObject flask11;
    public GameObject flask12;
    public GameObject sip11;
    public ParticleSystem SmokeParticleSystem;
    public TMP_Text m_TextComponent;
    public void SmokeEvent1()
    {
        SmokeParticleSystem.Play();
        m_TextComponent.text = "ГOTOBO";
        StartCoroutine(WaitForFunction1());
    }
    public void Event2()
    {
        m_TextComponent.text = "ГOTOBO";
        StartCoroutine(WaitForFunction2());
    }
    public void Event3()
    {
        m_TextComponent.text = "ГOTOBO";
        StartCoroutine(WaitForFunction3());
    }
    public void Event4()
    {
        m_TextComponent.text = "ГOTOBO";
        StartCoroutine(WaitForFunction4());
    }
    public void Event5()
    {
        m_TextComponent.text = "ГOTOBO";
        StartCoroutine(WaitForFunction5());
    }
    public void Event6()
    {
```

```

        m_TextComponent.text = "ΓOTOBO";
        StartCoroutine(WaitForFunction6());
    }
    public void Event7()
    {
        m_TextComponent.text = "ΓOTOBO";
        StartCoroutine(WaitForFunction7());
    }
    public void Event8()
    {
        m_TextComponent.text = "ΓOTOBO";
        StartCoroutine(WaitForFunction8());
    }
    IEnumerator WaitForFunction8()
    {
        yield return new WaitForSeconds(5);
    }
    IEnumerator WaitForFunction7()
    {
        yield return new WaitForSeconds(5);
    }
    IEnumerator WaitForFunction6()
    {
        yield return new WaitForSeconds(5);
    }
    IEnumerator WaitForFunction5()
    {
        yield return new WaitForSeconds(5);
    }
    IEnumerator WaitForFunction4()
    {
        yield return new WaitForSeconds(5);
    }
    IEnumerator WaitForFunction3()
    {
        yield return new WaitForSeconds(5);
    }
    IEnumerator WaitForFunction2()
    {
        yield return new WaitForSeconds(5);
    }
    IEnumerator WaitForFunction1()
    {
        yield return new WaitForSeconds(5);
        m_TextComponent.text = "*****";
    }

```

```

        flask11.SetActive(false);
        flask12.SetActive(false);
        sip11.SetActive(false);
    }
}

```

Файл «Liquidsim.cs»

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class Liquidsim : MonoBehaviour
{
    public float liquidLevel; // Уровень жидкости (от 0 до 1)
    public float liquidSpeed = 0.5f; // Скорость изменения уровня жидкости
    public float flaskheight = 1f; // Высота колбы

    private Renderer liquidRenderer;
    private Renderer particleliquidRenderer;

    private LiquidTransfer LiquidTransfer;
    //private Liquid Liquid;

    private List<Liquid> flaskLiquids;
    private Color a = new Color(0.8f, 0.8f, 0.8f, 0.8f);
    private Color b = new Color(0.1f, 0.5f, 0.1f, 0.8f);
    private Color c = new Color(0.8f, 0f, 0f, 0.8f);
    private Color uniquecolor;
    private Dictionary<string, Color> colors;

    private void Start()
    {
        LiquidTransfer =
        gameObject.transform.parent.gameObject.GetComponent<LiquidTransfer>();
        //Liquid =
        gameObject.transform.parent.gameObject.GetComponent<Liquid>();
        flaskLiquids = LiquidTransfer.flaskLiquids;
        liquidRenderer =
        gameObject.transform.parent.gameObject.GetComponent<Renderer>();
        particleliquidRenderer =
        gameObject.transform.parent.GetChild(2).GetComponent<Renderer>();
        liquidLevel = LiquidTransfer.flaskLiquidAmount;
        colors = new Dictionary<string, Color>()
        {
            { "K2CO3", a },
            { "0.25NH3", b },

```



```

        { "HCl", c}
    };
    if (liquidLevel != 0)
        uniquecolor = colors[LiquidTransfer.flaskLiquids[0].name];
    else
        uniquecolor = new Color(1, 1, 1, 0);
    liquidRenderer.material.SetColor("_Color", uniquecolor);
}

private void Update()
{
    // Вычисляем новую высоту жидкости
    float liquidFillHeight = liquidLevel * flaskheight;

    // Обновляем размеры и положение отображения жидкости
    if (liquidLevel != 0)
        SetLiquidColor();
}

public void SetLiquidColor()
{
    float r = 0f;
    float g = 0f;
    float b = 0f;
    foreach (Liquid liquid in flaskLiquids)
    {
        if (liquid != null)
            continue;
        r += colors[liquid.name].r * liquid.amount /
LiquidTransfer.maxLiquidVolume;
        g += colors[liquid.name].g * liquid.amount /
LiquidTransfer.maxLiquidVolume;
        b += colors[liquid.name].b * liquid.amount /
LiquidTransfer.maxLiquidVolume;
    }
    uniquecolor = new Color(r, g, b, 0.8f);
    particleliquidRenderer.material.SetColor("_Color", uniquecolor);
}
}

```

Файл «LiquidTransfer.cs»

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.Events;

public class LiquidTransfer : MonoBehaviour

```

```

{
    // Компоненты колб
    public GameObject flask1;
    private GameObject flask2;

    //Компоненты горлышек колб
    public GameObject flask1Neck;
    private GameObject flask2Neck;

    // Количество жидкости в колбе 1
    public List<Liquid> flaskLiquids = new List<Liquid>();

    private List<Recipe> recipes = new List<Recipe>();

    //Компоненты жидкостей колб
    public GameObject flask1Liquid;
    private GameObject flask2Liquid;

    //Начальная жидкость
    public string liquidname;
    public float liquidamount;

    //Скрипт второй колбы
    private LiquidTransfer flask2lt;

    //Система частиц для переливания
    public ParticleSystem liquidParticleSystem;
    // Количество жидкости в колбе 1
    public float flaskLiquidAmount;

    // Скорость переливания жидкости (единиц в секунду)
    private float transferSpeed = 0.1f;

    // Максимальный объем жидкости в колбах
    public float maxLiquidVolume = 1.0f;

    //Угол поворота для начала переливания
    public float angle = 90f;

    //создание финальной жидкости
    public UnityEvent finalEvent = new UnityEvent();

    void Start()
    {

```

```

liquidParticleSystem.Stop();
flaskLiquids.Add(new Liquid { name = liquidname, amount = liquidamount });
// Создаем список рецептов
recipes.Add(new Recipe("Дымная смесь", new List<string>() { "K2CO3",
"0.25NH3", "HCl" }));
recipes.Add(new Recipe("Смешанная кислота 2", new List<string>() { "Вода",
"Азотная кислота" }));
}

void Update()
{
    flaskLiquidAmount = GetTotalLiquidAmount(flaskLiquids);
    if (IsFlask1Tilted())
    {
        if (flaskLiquids.Count > 0)
        {
            // Выполняем лучи из колбы 1 вниз для обнаружения объекта под
колбой

            float liquidTransferAmount = transferSpeed * Time.deltaTime;
            float transferAmount = Mathf.Min(liquidTransferAmount,
GetTotalLiquidAmount(flaskLiquids));
            string liquid = TransferLiquid(flaskLiquids, transferAmount);
            liquidParticleSystem.Play();
            flask1Liquid.GetComponent<LiquidSim>().liquidLevel =
GetTotalLiquidAmount(flaskLiquids) / maxLiquidVolume;
            RaycastToTarget(liquid);
        }
        else if (flaskLiquidAmount == 0.0f)
            liquidParticleSystem.Stop();
    }
    CreateNewLiquid();
}
// Проверяет, что верхняя колба повернута набок
bool IsFlask1Tilted()
{
    // Получение значения поворота колбы 1 по оси x
    float rotationX = flask1.transform.rotation.eulerAngles.x;
    // Проверка, что колба 1 повернута набок
    if (rotationX < (270-angle) || rotationX > (270+angle))
        return true;
    return false;
}
// Выполняет лучи из колбы 1 вниз для обнаружения объекта под колбой
void RaycastToTarget(string liquid)

```

```

{
    // Изначально сбрасываем цель
    flask2 = null;

    // Выполняем луч из колбы 1 вниз
    RaycastHit hit;
    if (Physics.Raycast(flask1Neck.transform.position, -Vector3.up, out hit))
    {
        // Проверяем, что объект под колбой 1
        flask2Neck = hit.collider.gameObject;
        if (flask2Neck.tag != "flask")
        {
            flask2Neck = null;
        }
        else
        {
            Transfer(liquid);
        }
    }
}

void Transfer(string liquid)
{
    flask2 = flask2Neck.transform.parent.gameObject;
    flask2Liquid = flask2.transform.GetChild(0).gameObject;
    flask2lt = flask2.GetComponent<LiquidTransfer>();
    // Количество жидкости, которое можно перелить за текущий кадр
    float liquidTransferAmount = transferSpeed * Time.deltaTime;

    // Переливание жидкости из колбы 1 в колбу 2
    if (GetTotalLiquidAmount(flask2lt.flaskLiquids) < flask2lt.maxLiquidVolume)
    {
        float transferAmount = Mathf.Min(liquidTransferAmount,
            GetTotalLiquidAmount(flaskLiquids), flask2lt.maxLiquidVolume -
            GetTotalLiquidAmount(flask2lt.flaskLiquids));
        TransferLiquid2(flask2lt.flaskLiquids, transferAmount, liquid);
        // Обновление высоты жидкости в колбах
        flask2Liquid.GetComponent<Liquidsim>().liquidLevel =
            GetTotalLiquidAmount(flask2lt.flaskLiquids) / flask2lt.maxLiquidVolume;
    }
}

// Получает общее количество жидкости в списке жидкостей
float GetTotalLiquidAmount(List<Liquid> liquids)
{
    float totalAmount = 0.0f;
    foreach (Liquid liquid in liquids)

```

```

    {
        totalAmount += liquid.amount;
    }
    return totalAmount;
}
// Переливает заданное количество жидкости из списка жидкостей
string TransferLiquid(List<Liquid> liquids, float amount)
{
    float remainingAmount = amount;
    for (int i = liquids.Count - 1; i >= 0; i--)
    {
        Liquid liquid = liquids[i];
        float transferAmount = Mathf.Min(remainingAmount, liquid.amount);
        liquid.amount -= transferAmount;
        remainingAmount -= transferAmount;

        // Удаляем жидкость, если ее количество стало нулевым
        if (liquid.amount <= 0.0f)
        {
            liquids.RemoveAt(i);
        }
        // Прерываем цикл, если перелито достаточное количество жидкости
        if (remainingAmount <= 0.0f)
        {
            return liquid.name;
        }
    }
    return null;
}
void TransferLiquid2(List<Liquid> liquids, float amount, string liquid2name)
{
    float remainingAmount = amount;
    int indexliquid2 = -1;
    float transferAmount = Mathf.Min(remainingAmount, flask2lt.maxLiquidVolume
- GetTotalLiquidAmount(flask2lt.flaskLiquids));
    for (int i = 0; i < liquids.Count; i++)
    {
        if (liquids[i].name == liquid2name)
        {
            indexliquid2 = i;
            break;
        }
    }
    if (indexliquid2 != -1)
    {

```

```

        liquids[indexliquid2].amount += transferAmount;
        remainingAmount -= transferAmount;
    }
    else
    {
        liquids.Add(new Liquid { name = liquid2name, amount = transferAmount });
    }
}
void CreateNewLiquid()
{
    // Проверяем наличие двух или более жидкостей в колбе 1
    if (flaskLiquids.Count >= 2)
    {
        // Проверяем каждый рецепт
        foreach (Recipe recipe in recipes)
        {
            // Проверяем, есть ли нужные жидкости для данного рецепта в колбе 1
            if (HasRequiredLiquids(recipe, flaskLiquids))
            {
                int indexresliq = -1;
                for (int i = 0; i < flaskLiquids.Count; i++)
                {
                    if (flaskLiquids[i].name == recipe.resultName)
                    {
                        indexresliq = i;
                        break;
                    }
                }
                if (indexresliq == -1)
                {
                    // Создаем новую жидкость с заданным именем
                    Liquid newLiquid = new Liquid();
                    newLiquid.name = recipe.resultName;
                    indexresliq = flaskLiquids.Count;
                    flaskLiquids.Add(newLiquid);
                }

                // Считаем общее количество жидкости в колбе 1
                float totalAmount = GetTotalLiquidAmount(flaskLiquids);

                float minliq = totalAmount;

                // Переносим жидкости по рецепту в новую жидкость
                foreach (string requiredLiquidName in recipe.requiredLiquids)
                {

```

```

        int requiredLiquid = flaskLiquids.FindIndex(liquid => liquid.name ==
requiredLiquidName);
        if (flaskLiquids[requiredLiquid].amount < minliq)
            minliq = flaskLiquids[requiredLiquid].amount;
    }
    foreach (string requiredLiquidName in recipe.requiredLiquids)
    {
        int requiredLiquid = flaskLiquids.FindIndex(liquid => liquid.name ==
requiredLiquidName);
        flaskLiquids[indexresliq].amount += minliq;
        flaskLiquids[requiredLiquid].amount -= minliq;
    }
    // Добавляем новую жидкость в колбу 1
    proverka(flaskLiquids[indexresliq]);
}
}
}
}

```

// Проверяет, есть ли нужные жидкости для рецепта в списке жидкостей
bool HasRequiredLiquids(Recipe recipe, List<Liquid> liquids)

```

{
    foreach (string requiredLiquidName in recipe.requiredLiquids)
    {
        if (!liquids.Exists(liquid => liquid.name == requiredLiquidName))
        {
            return false;
        }
    }
    return true;
}
void proverka(Liquid newLiquid)
{
    if (newLiquid.name == "Дымная смесь" && newLiquid.amount > 0.5)
        finalEvent?.Invoke();
}
}
}

```

// Класс, представляющий жидкость

[System.Serializable]

public class Liquid

```

{
    public string name;        // Название жидкости
    public float amount;       // Количество жидкости
}

```

```

}

// Класс, представляющий рецепт
[System.Serializable]
public class Recipe
{
    public List<string> requiredLiquids; // Список требуемых жидкостей
    public string resultName;           // Имя результирующей жидкости
    public Recipe(string result, List<string> liquids)
    {
        requiredLiquids = new List<string>(liquids);
        resultName = result;
    }
}

```

Файл «menucontroller.cs»

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;

public class menucontroller : MonoBehaviour
{
    public void StartBtn()
    {
        SceneManager.LoadScene("MainScene");
    }
    public void ExitBtn()
    {
        Application.Quit();
    }
}

```


3.2. Рисунки моделей

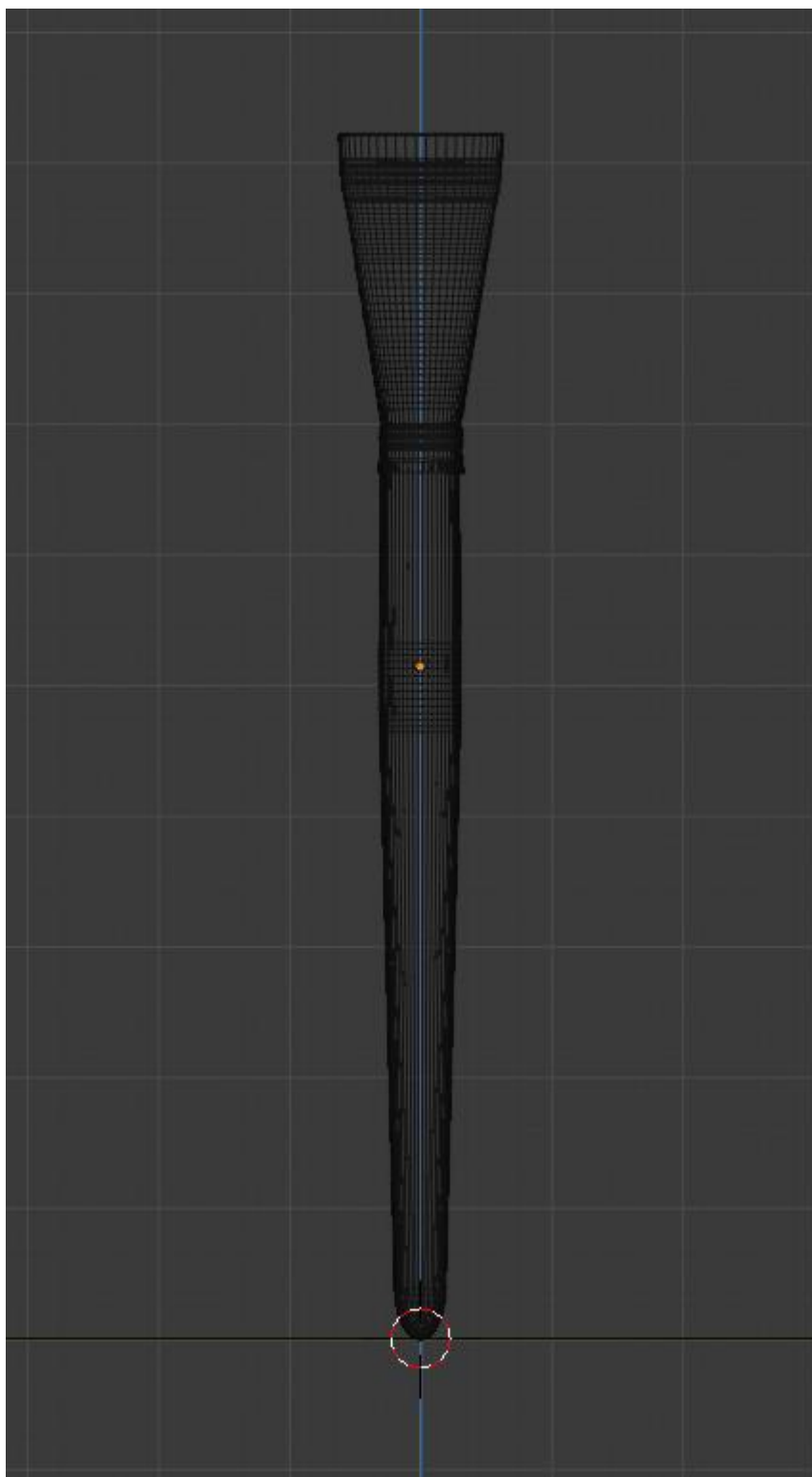


Рисунок 1 – Модель кисточки

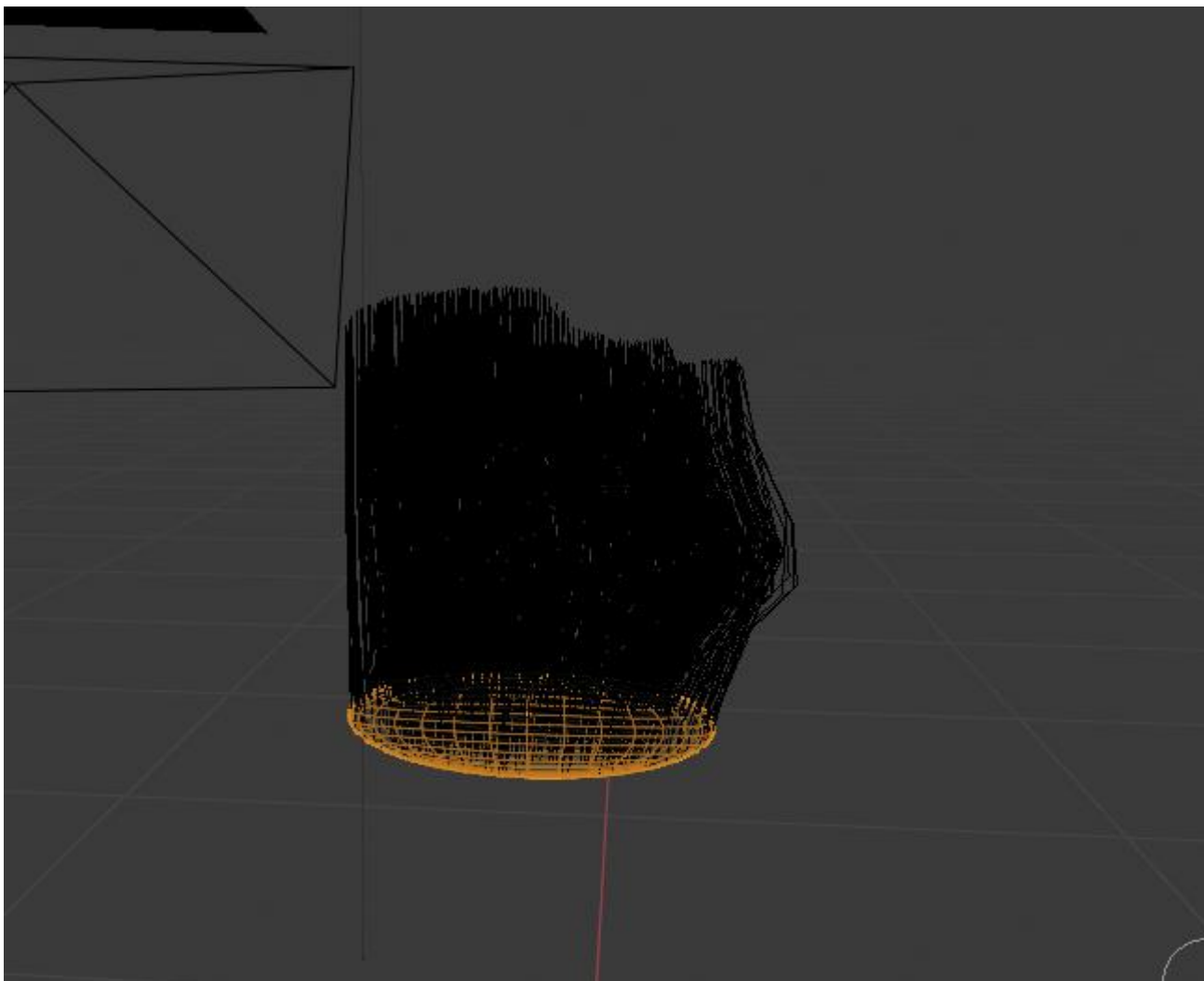


Рисунок 2 – Модель волос кисточки

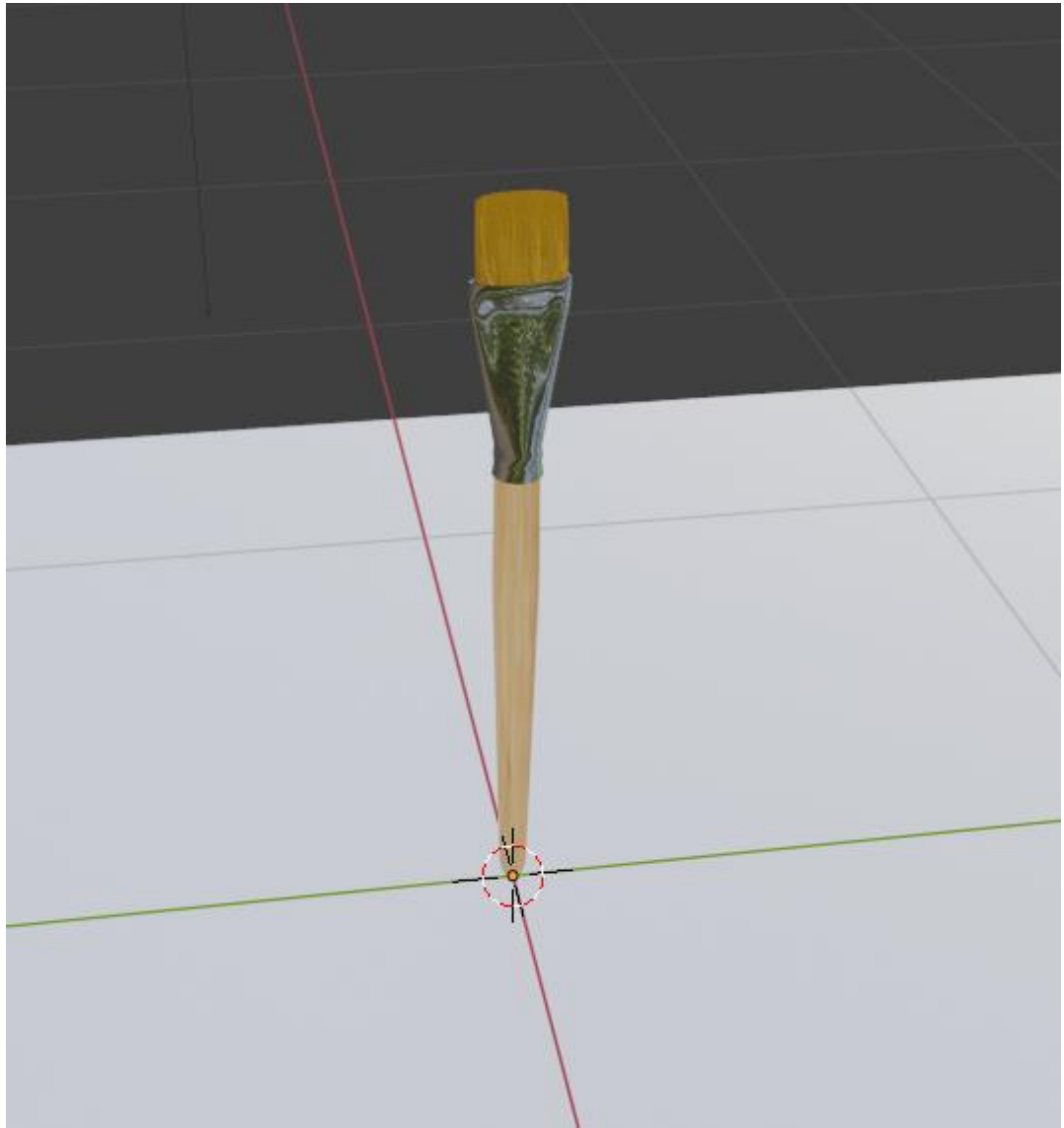


Рисунок 3 – Полная модель кисточки

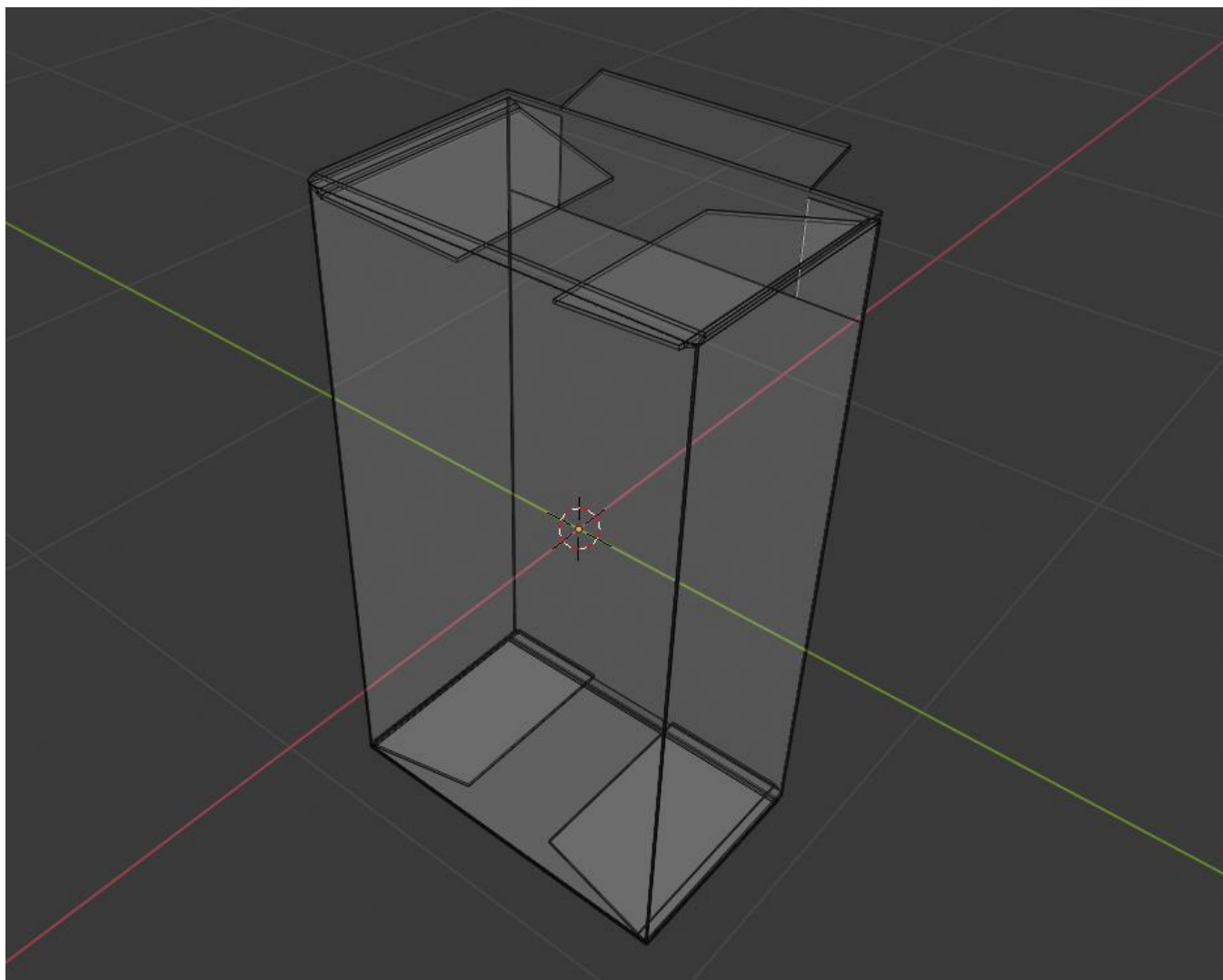


Рисунок 4 – Модель упаковки соды

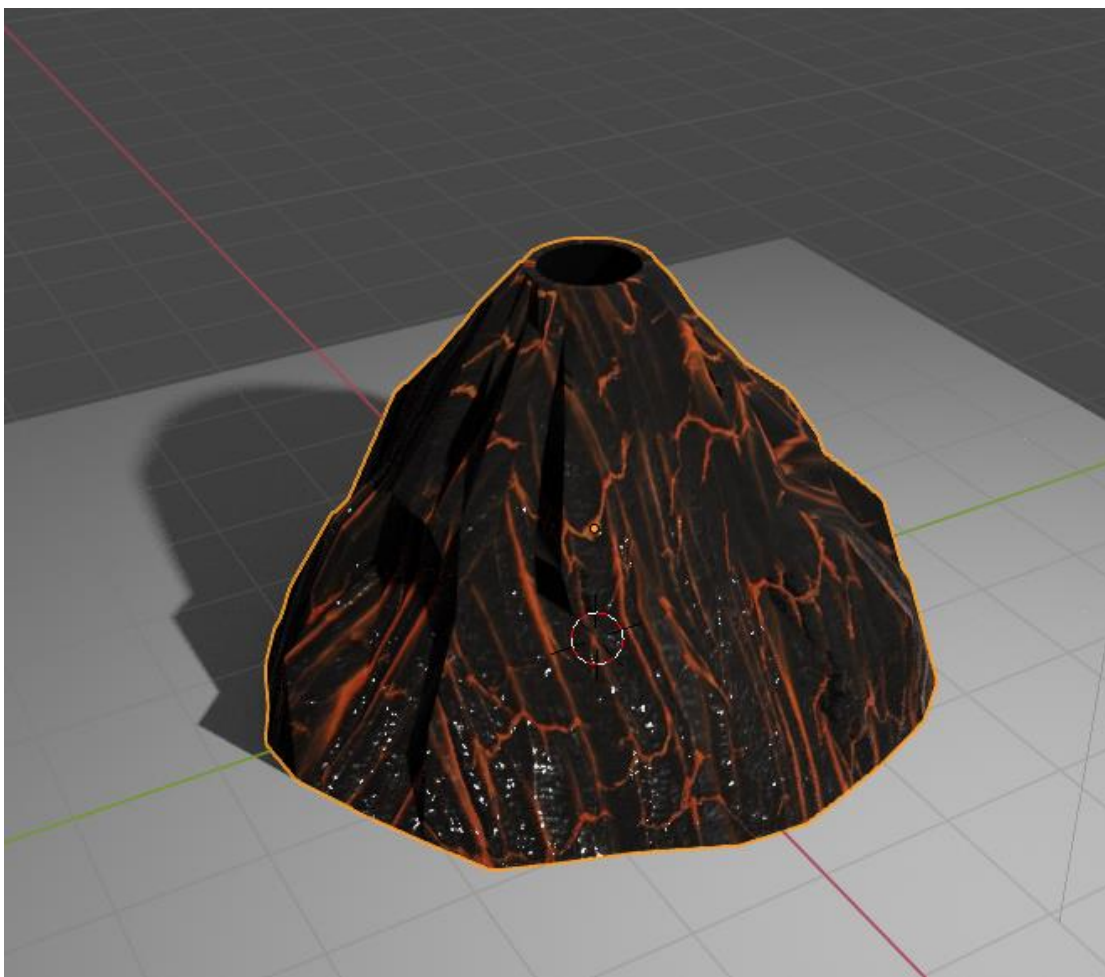


Рисунок 5 – Модель вулкана

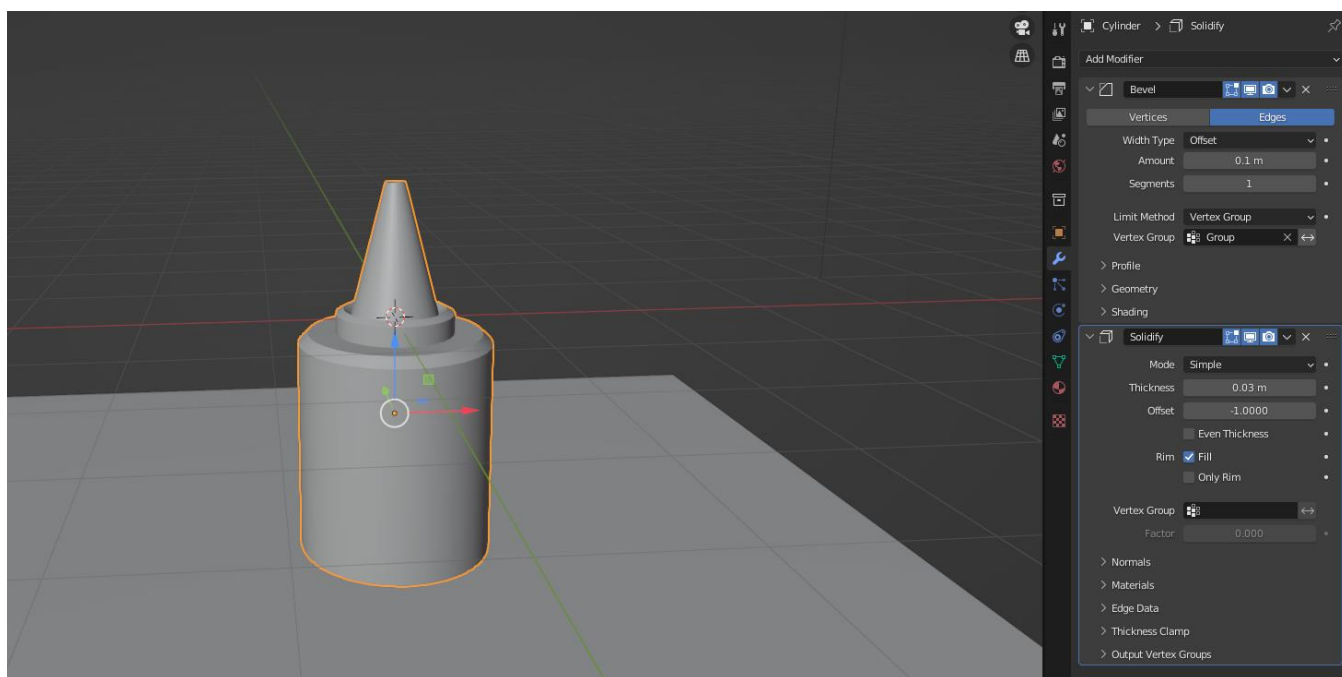


Рисунок 6 – Модель банки с носиком

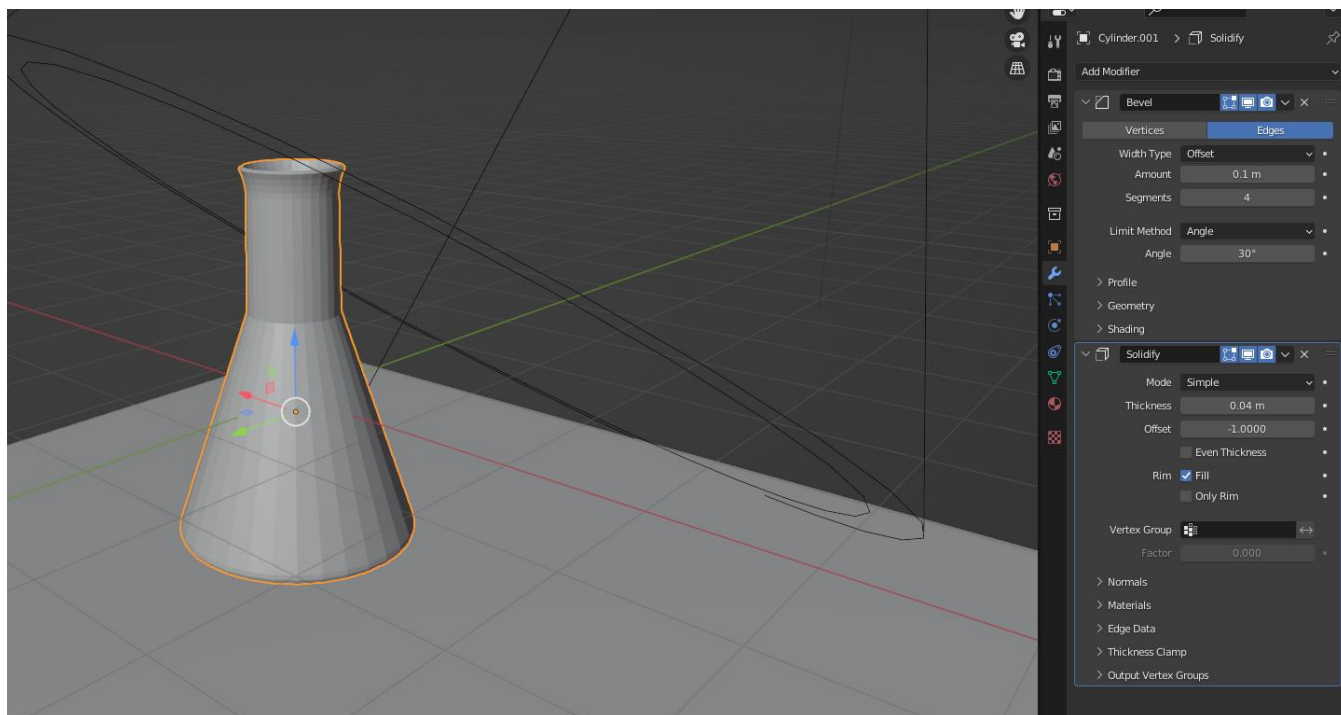


Рисунок 7 – Модель колбы

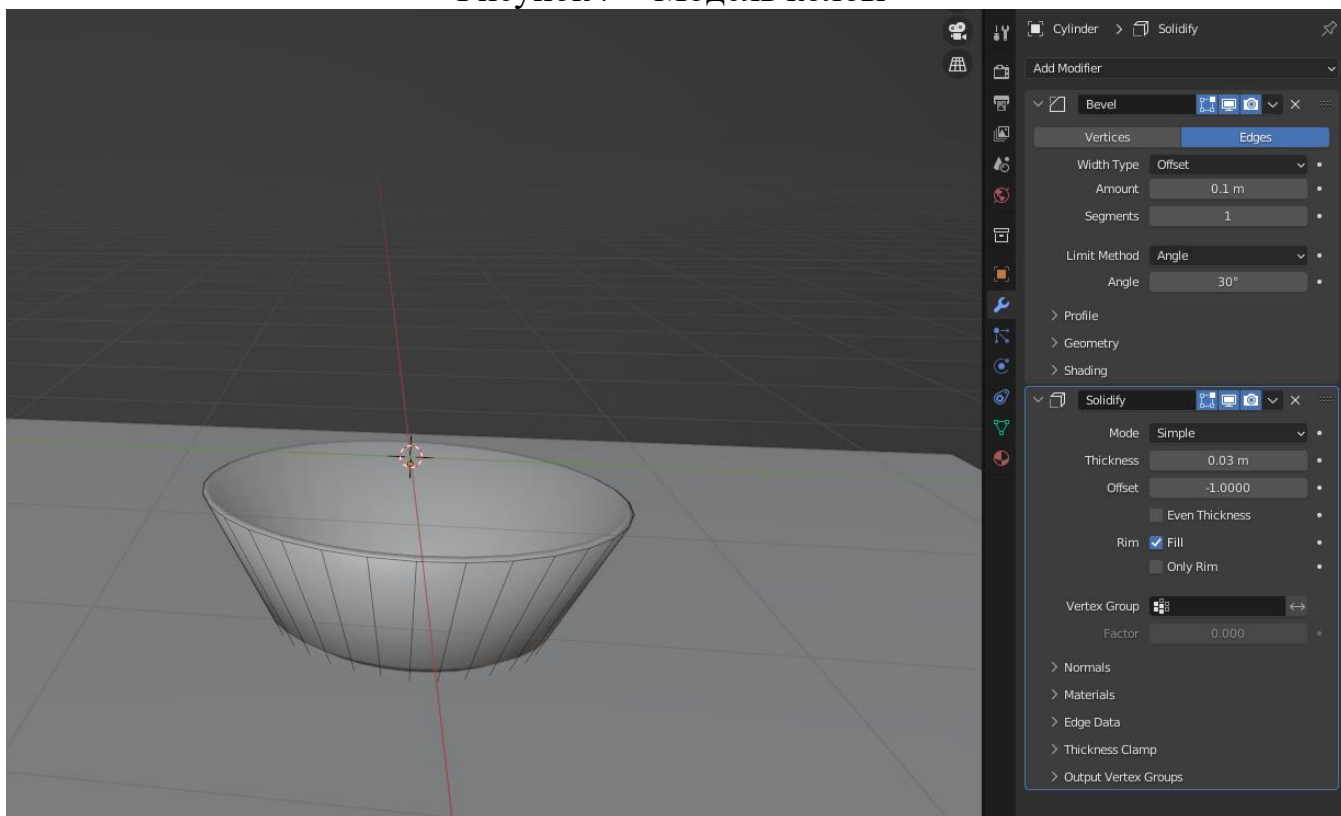


Рисунок 8 – Модель тарелки

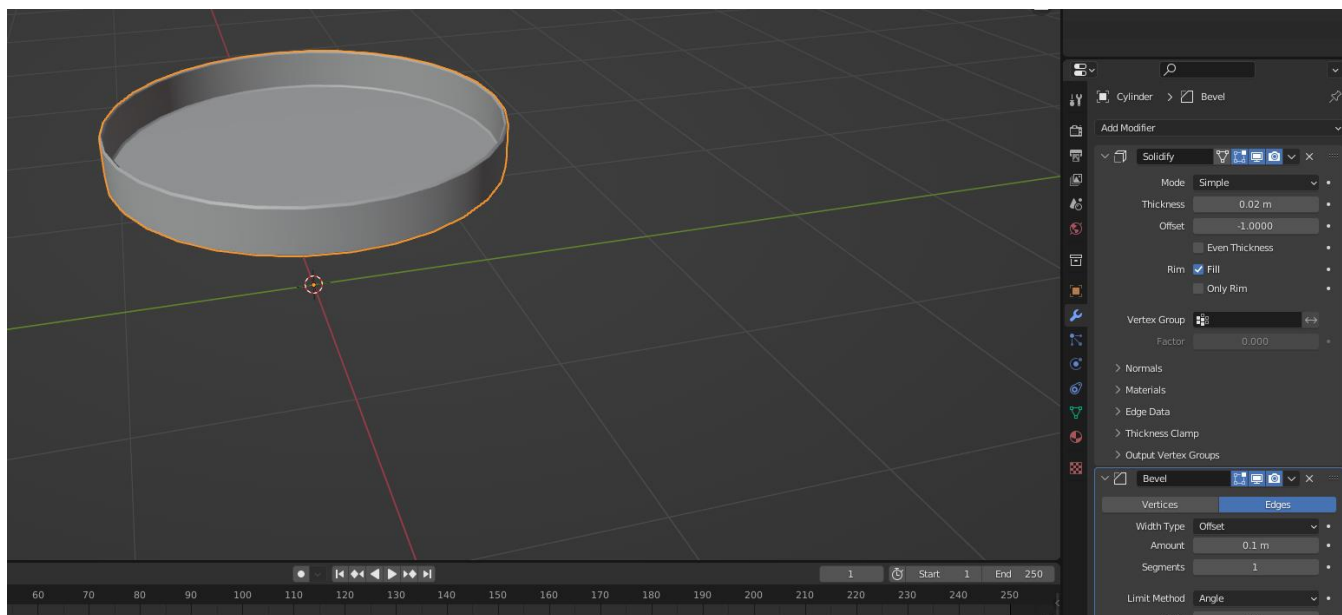


Рисунок 9 – Модель чашки Петри

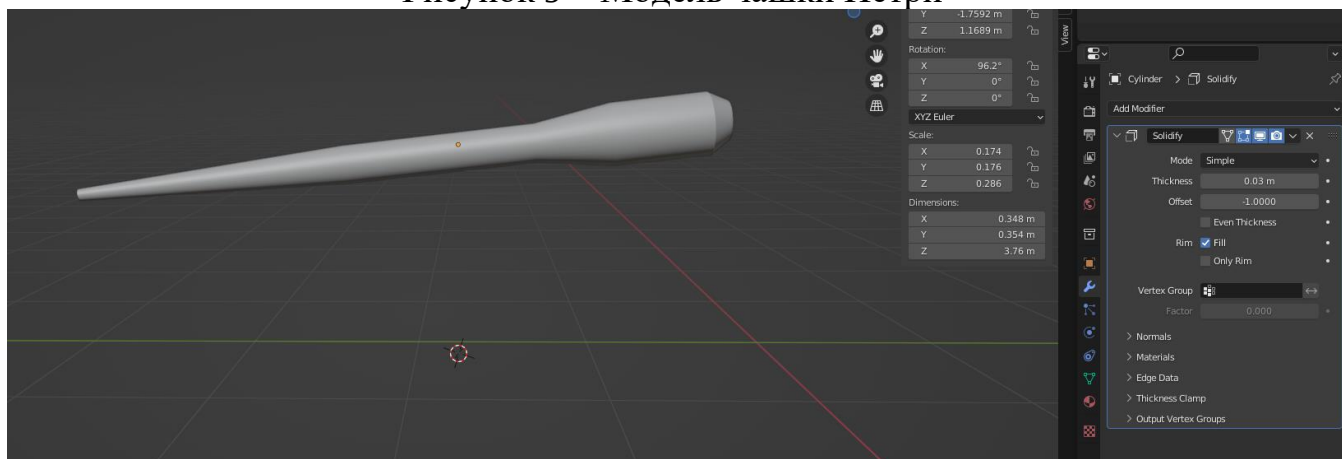


Рисунок 10 – Модель пипетки

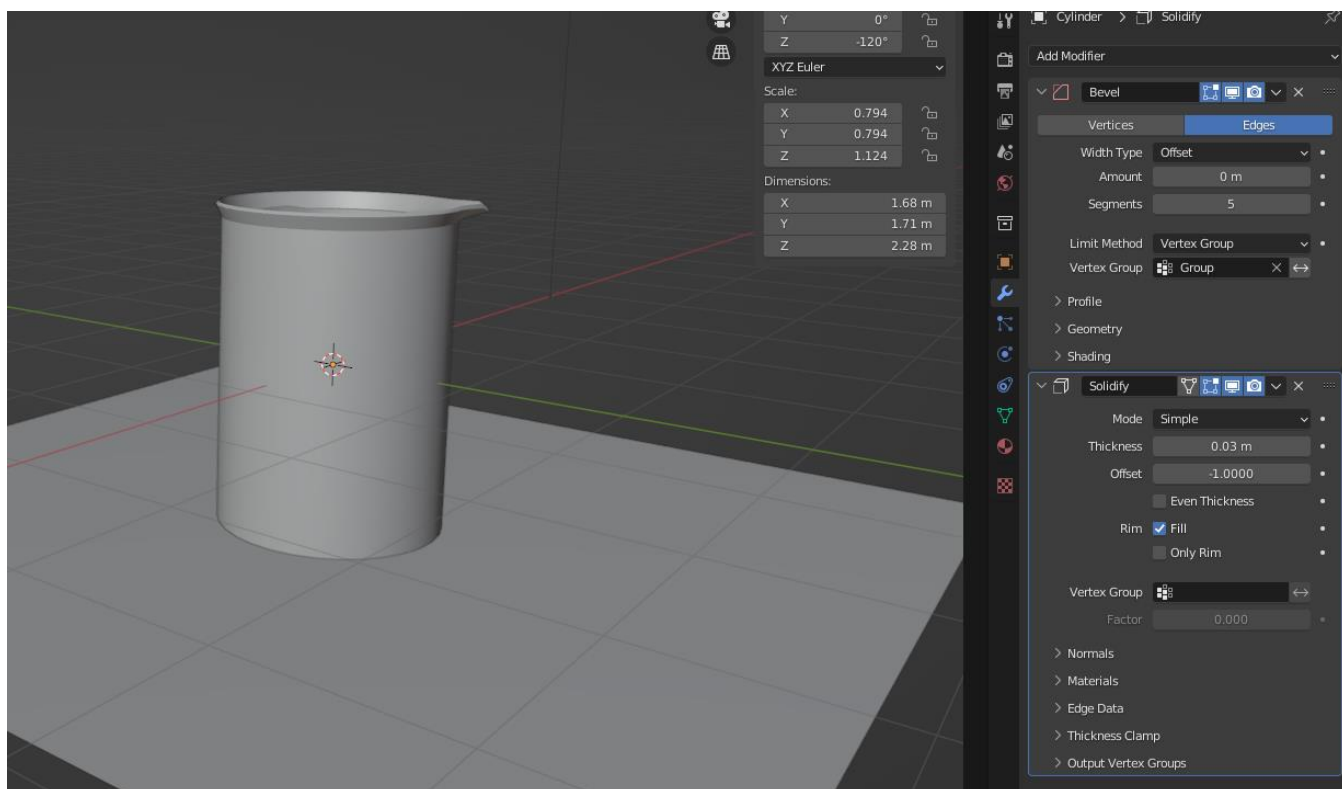


Рисунок 11 – Модель стакана

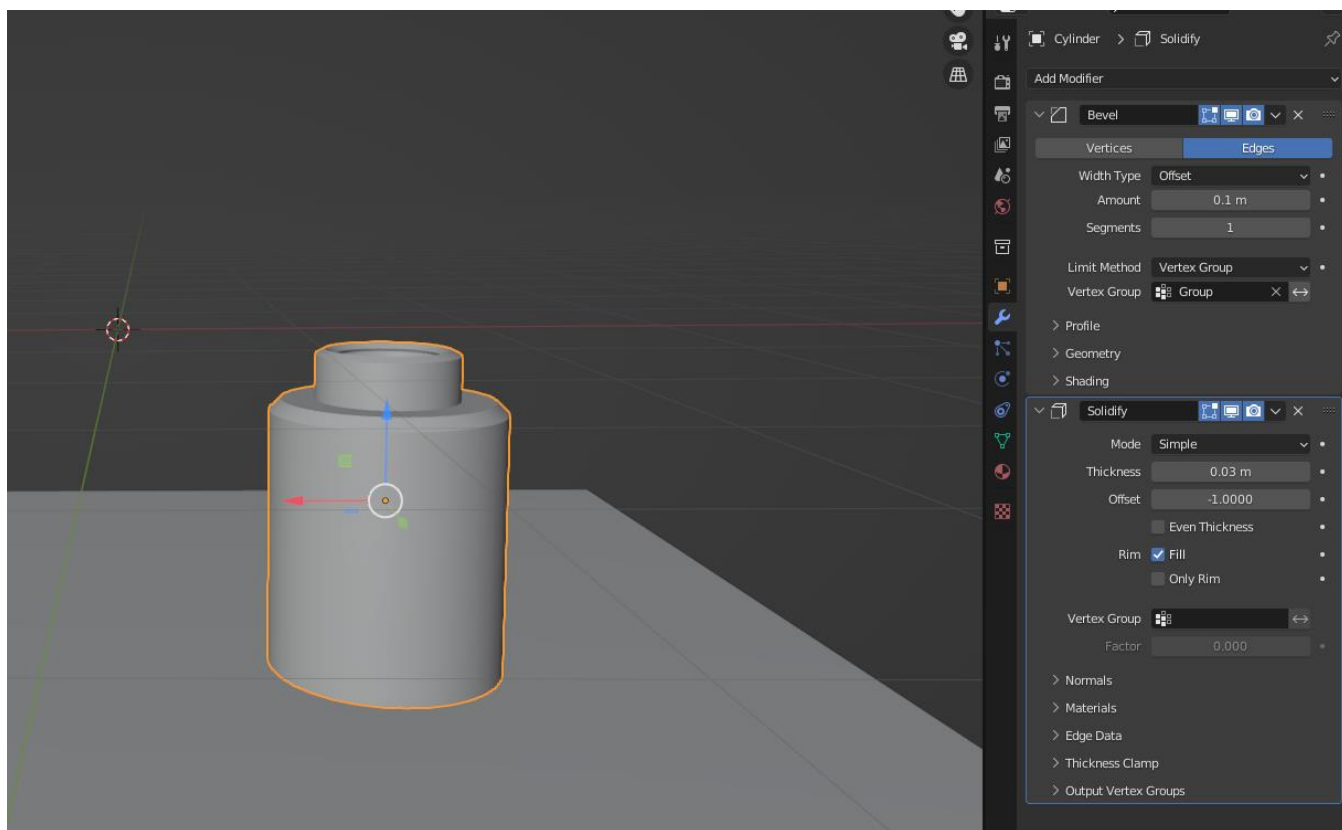


Рисунок 12 – Модель банки для сыпучки



Рисунок 13 – Модель шпателя

Аннотация

В документе указаны общие сведения о VR-программе «Практический курс по химии». Приведено общее описание функционирования программы.. Описаны исходные и выходные параметры и логика работы программы, даны сведения об используемых технических средствах и запуске программы.

СОДЕРЖАНИЕ

- 4.1. Общие сведения
- 4.2. Функциональное назначение
- 4.3. Описание логической структуры
- 4.4. Используемые технические средства
- 4.5. Вызов и загрузка

4.1. Общие сведения

Программа написана на языке C# и игровом движке Unity 3D. VR-программа «Практический курс по химии» — это игровая программа, где предоставляется возможность в режиме виртуальной реальности (VR) выполнить химические эксперименты из школьной химии: Пользователь находится в лабораторном кабинете, где возможно без рисков для здоровья совершать ошибки и работать с труднодоступными реагентами. Для функционирования данной программы необходима ОС Windows 10 и комплект VR Oculus.

4.2. Функциональное назначение

С помощью данной программы пользователь может попробовать неограниченное количество раз провести химические опыты. Данное приложение предназначено для лиц любой возрастной категории, является развлекательным и познавательным.

4.3. Описание логической структуры

Программа написана на языке C#. При открытии программы появляется окно с меню игры. Для продолжения работы пользователь может перейти к меню выбора опыта. Далее пользователь должен нажать кнопку «Старт», после чего начинается основной игровой процесс.

4.4. Используемые технические средства

Для нормального функционирования программы необходимо наличие ПК, на который установлена ОС Windows 10 и есть DirectX 10, а так же VR комплекта Oculus.

4.5. Вызов и загрузка

Вызов и загрузка программы осуществляется в следующей последовательности: Запуск исполняемого. Дальнейшие действия определяются соответствующими элементами управления.

44.4143.22-31

СТУДЕНТЫ ГР.	<u>4143</u>				<u>В. Д. Алексеева</u>
					<u>А. М. Гридин</u>
					<u>П. В. Суслов</u>
					<u>С. А. Шалев</u>
				подпись, дата	

подпись, дата

39

Аннотация

В документе приводится назначение программы, её основные характеристики и область применения. Также приводятся общие характеристики входной и выходной информации, описывается метод решения задачи.

СОДЕРЖАНИЕ

- 5.1. Назначение программы
- 5.2. Условие применения
- 5.3. Описание задачи
- 5.4. Характеристика занимаемой памяти

5.1. Назначение программы

VR-Программа «Практический курс по химии» - программа, предназначенная для имитации лабораторных опытов. С помощью данной программы пользователь может попробовать неограниченное количество раз смешивать реагенты, наблюдать химические реакции. Данное приложение предназначено для лиц любой возрастной категории и является развлекательным.

5.2. Условия применения

Для нормального функционирования программы необходимо наличие ПК, на котором установлена ОС Windows 10 и выше, а так же VR комплект Oculus. Технические средства должны обеспечить вывод результатов выполнения в шлем. Для запуска программы необходимо запустить исполняемый файл.

5.3 Описание задачи

Разработать программу, реализующую VR-программу «Практический курс по химии» в режиме пользователь-компьютер. Пользователю должно быть доступно управление игровым процессом, который включает в себя:

- выбор опыта;
- взаимодействие с предметами;
- оценка действий игрока.

Программа должна являться готовым продуктом и соответствовать требованиям технического задания:

- легкодоступный и понятный интерфейс;
- локализация информации в игре;
- наличие трёхмерной графики.

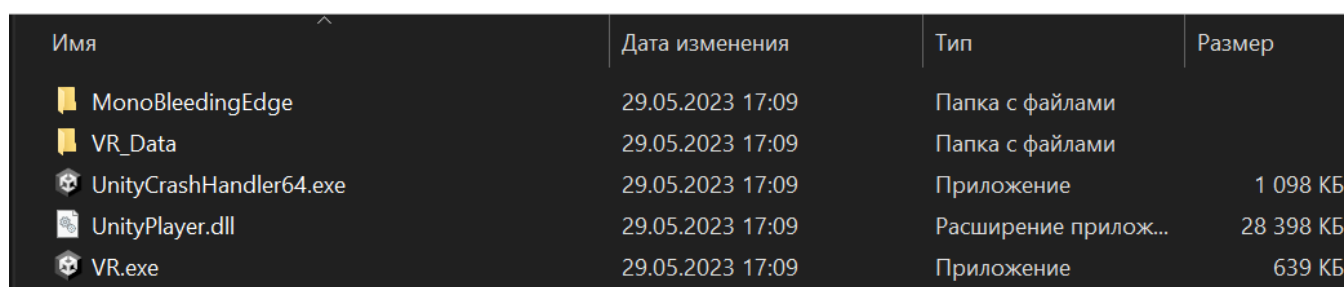
Основной игровой процесс должен включать в себя:

- взаимодействие с объектами;
- выполнение работ;
- своевременное уведомление игрока об ошибках;
- оценку действий и результат экзамена.

5.4 Характеристики занимаемой памяти

Название	Тип	Занимаемая память
Код	*.cs	19 Кб
Текстуры	*.psd, *.png, *.jpg	160 Мб
Префабы	*.prefab	2.57 Мб
Сцены	*.unity	3.28 Мб
Модели	*.fbx	491 Мб
Исполняемый файл	*.exe (*.dll, и другие)	31 Мб

Таблица 1 – Характеристики занимаемой памяти



Имя	Дата изменения	Тип	Размер
MonoBleedingEdge	29.05.2023 17:09	Папка с файлами	
VR_Data	29.05.2023 17:09	Папка с файлами	
UnityCrashHandler64.exe	29.05.2023 17:09	Приложение	1 098 КБ
UnityPlayer.dll	29.05.2023 17:09	Расширение прилож...	28 398 КБ
VR.exe	29.05.2023 17:09	Приложение	639 КБ

Рисунок 14 – Программа в проводнике

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана программа «Практический курс по химии», которая позволяет пользователю провести лабораторные опыты по химии. Программа тестирована и полностью соответствует требованиям технического задания на курсовую работу.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 ГОСТ 7.32-2017 Межгосударственный стандарт. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления.

2 ГОСТ 19.201-78 Техническое задание. Требования к содержанию и оформлению.

3 ГОСТ 19.202-78 Спецификация. Требования к содержанию и оформлению.

4 ГОСТ 19.401-78 Текст программы. Требования к содержанию и оформлению.

5 ГОСТ 19.402-78 Описание программы. Требования к содержанию и оформлению.

6 ГОСТ 19.404-79 Пояснительная записка. Требования к содержанию и оформлению.

7 ГОСТ 19.502-78 Описание применения. Требования к содержанию и оформлению.

8 Документация Unity [Электронный ресурс], URL - <https://docs.unity3d.com/ru/530/Manual/UnityManual.html>, режим доступа свободный

9 Сайт с химическими опытами [Электронный ресурс], URL - <http://www.sev-chem.narod.ru/opyt.htm>, режим доступа свободный

10 Документация Blender 3.6 [Электронный ресурс], URL - <https://docs.blender.org/manual/ru/dev/>, режим доступа свободный